



PDF Download
3649217.3653574.pdf
28 January 2026
Total Citations: 53
Total Downloads: 2948



Published: 03 July 2024

[Citation in BibTeX format](#)

ITiCSE 2024: Innovation and Technology
in Computer Science Education
July 8 - 10, 2024
Milan, Italy

Conference Sponsors:
SIGCSE

Latest updates: <https://dl.acm.org/doi/10.1145/3649217.3653574>

RESEARCH-ARTICLE

Desirable Characteristics for AI Teaching Assistants in Programming Education

PAUL DENNY, University of Auckland, Auckland, AUK, New Zealand

STEPHEN MACNEIL, Temple University, Philadelphia, PA, United States

JAROMÍR ŠAVELKA, Carnegie Mellon University, Pittsburgh, PA, United States

LEO PORTER, University of California, San Diego, San Diego, CA, United States

ANDREW LUXTON-REILLY, University of Auckland, Auckland, AUK, New Zealand

Open Access Support provided by:

University of California, San Diego

Carnegie Mellon University

University of Auckland

Temple University

Desirable Characteristics for AI Teaching Assistants in Programming Education

Paul Denny
University of Auckland
Auckland, New Zealand
paul@cs.auckland.ac.nz

Stephen MacNeil
Temple University
Philadelphia, PA, United States
stephen.macneil@temple.edu

Jaromir Savelka
Carnegie Mellon University
Pittsburgh, PA, United States
jsavelka@cs.cmu.edu

Leo Porter
University of California, San Diego
San Diego, CA, United States
leporter@ucsd.edu

Andrew Luxton-Reilly
University of Auckland
Auckland, New Zealand
a.luxton-reilly@auckland.ac.nz

ABSTRACT

Providing timely and personalized feedback to large numbers of students is a long-standing challenge in programming courses. Relying on human teaching assistants (TAs) has been extensively studied, revealing a number of potential shortcomings. These include inequitable access for students with low confidence when needing support, as well as situations where TAs provide direct solutions without helping students to develop their own problem-solving skills. With the advent of powerful large language models (LLMs), digital teaching assistants configured for programming contexts have emerged as an appealing and scalable way to provide instant, equitable, round-the-clock support. Although digital TAs can provide a variety of help for programming tasks, from high-level problem solving advice to direct solution generation, the effectiveness of such tools depends on their ability to promote meaningful learning experiences. If students find the guardrails implemented in digital TAs too constraining, or if other expectations are not met, they may seek assistance in ways that do not help them learn. Thus, it is essential to identify the features that students believe make digital teaching assistants valuable. We deployed an LLM-powered digital assistant in an introductory programming course and collected student feedback ($n = 813$) on the characteristics of the tool they perceived to be most important. Our results highlight that students value such tools for their ability to provide instant, engaging support, particularly during peak times such as before assessment deadlines. They also expressed a strong preference for features that enable them to retain autonomy in their learning journey, such as scaffolding that helps to guide them through problem-solving steps rather than simply being shown direct solutions.

CCS CONCEPTS

• Social and professional topics → Computing education; • Computing methodologies → Artificial intelligence.

KEYWORDS

Feedback, LLMs, Digital TAs, Automated tutors, AI tutors

ACM Reference Format:

Paul Denny, Stephen MacNeil, Jaromir Savelka, Leo Porter, and Andrew Luxton-Reilly. 2024. Desirable Characteristics for AI Teaching Assistants in Programming Education. In *Proceedings of the 2024 Innovation and Technology in Computer Science Education V. 1 (ITiCSE 2024)*, July 8–10, 2024, Milan, Italy. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3649217.3653574>

1 INTRODUCTION

High-quality and timely feedback is essential in educational settings, particularly for students in large undergraduate programming courses who often have a wide variety of prior experience. Human teaching assistants (TAs) have helped address this need, but scaling their support to match classes that are growing in size and diversity remains a significant challenge. Prior literature on student interactions with TAs has revealed mismatches between student expectations and educational objectives. In particular, students grappling with impending deadlines tend to prioritize assignment completion over long-term learning and conceptual mastery [18]. In practice, TAs frequently reveal solutions during help sessions instead of guiding students through the debugging process, further entrenching problem-solving dependencies rather than fostering independence and resilience [15]. Another limitation of human TAs is their availability and accessibility. Providing immediate help in programming courses is difficult, with scheduled office hours being underutilized [34] and in-person support failing to reach all students equally, as some may hesitate to seek assistance while others monopolize it [33].

The scalability and performance of large language models (LLMs) has prompted considerable interest in their use for powering digital teaching assistants, offering much higher availability and faster response times than human TAs. However, a critical consideration for instructors is the ability of digital TAs to support healthy learning interactions. There are concerns that students may become overly dependent on these tools for solutions. This calls for built-in “guardrails” to prevent such misuse [6]. However, if the constraints imposed by such guardrails are overly prohibitive, or a digital TA lacks some other desired characteristics, then students may instead turn to publicly available unconstrained LLMs which may not be as pedagogically useful. Presently, very little is known about the



This work is licensed under a Creative Commons Attribution International 4.0 License.

ITiCSE 2024, July 8–10, 2024, Milan, Italy
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0600-4/24/07.
<https://doi.org/10.1145/3649217.3653574>

specific characteristics that students value in digital TAs, and how these might differ from their expectations of human tutors, despite the fact that such tools are likely to become ubiquitous.

To address this gap, we deployed a digital TA in a large undergraduate introductory programming course, in order to assess students’ perceptions and preferences. As part of a structured lab activity, students observed LLM-generated responses to their own queries for debugging and code writing assistance on sample problems. After the lab, students were also optionally able to use the digital TA to seek assistance for their own project work. Students evaluated the quality of the feedback produced by the digital TA to their queries, and also reflected on the characteristics that they felt were most important in a digital TA. This research focuses on two central questions regarding digital TA deployment:

RQ1: How do students perceive the effectiveness of a digital TA’s responses concerning debugging and code writing tasks in terms of correctness and helpfulness?

RQ2: What attributes of a digital programming assistant do students consider most essential for supporting their learning in introductory programming courses?

2 RELATED WORK

2.1 LLMs in Computing Education

LLMs have been extensively studied in the context of computing education [25]. There is a growing body of work focusing on the capabilities of LLMs for solving programming exercises [5, 7, 9, 30], as well as answering [29] and generating programming-related MCQs [35]. Other studied tasks include bug detection [19], test suite authoring [4], automatic generation of programming exercises [28], code explanations [16, 20], and worked examples [12], and the ways that students interact with code-generating LLMs when programming [13, 26]. When employed as digital TAs, it is not clear if LLMs are as effective as human tutors [22, 29]. For example, a study in the context of mathematics education by Pardos and Bhandari found that significant learning gains were only observed for students who were provided human-created hints, and not for those who were given LLM-generated hints [24]. There remains much ongoing discussion and work exploring the quality and completeness of feedback generated by LLMs in computing education contexts [2, 8].

2.2 Student and TA interactions in CS

Prior work has shown that student and TA interactions are challenging for both students and tutors. Krause-Levy et al. examined interactions between students and undergraduate TAs during help-seeking sessions focused on debugging assignments in introductory computing courses [15]. They found that the TAs often gave students the solution without teaching them the underlying concepts of the process of debugging. This behavior results in students leaving tutoring sessions with resolved issues but potentially without a deeper understanding of how to independently tackle similar problems in the future.

Human TAs face challenges providing effective help for students because of a need to triage and prioritize multiple student requests, engage students effectively, and balance teaching with time constraints [21, 27]. In addition, TAs must deal with their own

understanding of the material, unprepared students, and diagnosing student knowledge to provide appropriate assistance [27].

Most directly related to this work is that of Lim et al. [18] who explored student expectations of human TAs. The main findings from interviews with students in a CS2 course indicate that student expectations and desires can be misaligned with their learning goals, particularly under the pressure of imminent assignment deadlines. While students acknowledge the importance of understanding and learning from tutors, many prefer direct answers or solutions as deadlines approach to prioritize completing assignments over deeper learning. Additionally, students do not always value the importance of identifying their own bugs and may expect tutors to do it for them, potentially creating a reliance on tutors and compromising long-term learning.

3 METHODS

3.1 Course context and LLM

Our data was collected from a large introductory programming course taught at the University of Auckland, a large public research university in New Zealand, in 2023. A total of 889 students were enrolled in the course which spans a 12-week teaching term. Every week, students complete a set of laboratory tasks each contributing 2% towards their final grade in the course.

For the purposes of the current study, we provided students access to an LLM-powered digital assistant designed for programming courses called CodeHelp [17]. CodeHelp is open source¹ and is one of several applications in a publicly available framework that can be used for building web applications that use LLMs for educational purposes. Although CodeHelp is open source, an OpenAI API key must be provided so that it can make requests to one of OpenAI’s LLMs. For this study we chose to use GPT-4, which was the most capable model at the time but also the most expensive. The total cost for running the study over 13 days (9–21 October 2023) was US\$510.

CodeHelp is designed to answer free-form student inquiries while maintaining a key safeguard: it does not generate solution code, irrespective of the query’s phrasing. This is achieved with a series of distinct prompts which include explicit instructions not to generate code, and which separately verify that code is absent in any responses received from the LLM. This design is intended to allow students to receive step-by-step guidance in a mixture of natural language and pseudocode, as well as explanations of ideas to develop conceptual understanding, but require that they construct syntactically correct code on their own. CodeHelp also uses prompts to generate responses that are “positive and encouraging”, much like a human tutor would be expected to. Examples of this tone can be seen in the last line of the outputs shown in Figures 2 and 5.

3.2 Lab tasks

To ensure that all students had a comparable experience with CodeHelp, we prepared three tasks that were included on the lab (Week 11). These tasks consisted of one *code writing* and two *debugging* problems. For each task students were provided some sample code and were asked to construct an appropriate query for CodeHelp.

¹<https://github.com/liffiton/Gen-Ed>

For this task, you will explore the feedback generated by LLM-TA if it is asked to *write code*. Below is a short (incomplete) code snippet for a function that should test whether or not a number is a prime number (i.e. a positive integer which has exactly two distinct divisors: 1 and itself).

```
int IsPrime(int value)
{
    int i;
    return 0;
}
```

Copy the code above (i.e. the incomplete function) and provide it as input to LLM-TA. You should construct a short natural language description asking it to *write the code for this function*.

Submit the code and question to LLM-TA, and read the response that it generates.

Figure 1: Task 1 – writing code for the IsPrime() function

Each task was followed by two evaluation questions where students rated the correctness and helpfulness of the feedback produced by CodeHelp. Figures 1, 3 and 4 show the three tasks.

Code writing assistance: Task 1 involved *code writing*, and required students to provide the code fragment (for the incomplete IsPrime function in Figure 1) to CodeHelp and construct a query asking it to complete the code. Submissions for Task 1 tended to be relatively short, as exemplified by the following queries:

- *Could you please write me a function that determines whether an integer is a prime number or not?*
- *Can you write the code for this function?*

One example response to the query “Can you write the code for this function?”, when the code from Figure 1 was included verbatim, is shown in Figure 2.

Sure, I can help you understand how to write a function to check if a number is prime in C. A prime number is a natural number greater than 1 that has no positive divisors other than 1 and itself. The first few prime numbers are 2, 3, 5, 7, 11, and so on. To check if a number is prime, you can follow these steps:

1. If the number is less than 2, it's not prime. So, you can return 0.

<Steps 2 and 3 not shown for space reasons>

4. If the number is not divisible by any number in the above step, then the number is prime. You can return 1.

Remember to use a loop for step 3. The reason we only need to check divisibility up to the square root of the number is because a larger factor of the number would be a multiple of a smaller factor that has already been checked.

I hope this helps! Let me know if you have any other questions.

Figure 2: Example response from CodeHelp to Task 1.

Debugging assistance: The other two tasks were *debugging* tasks, where students were asked to query CodeHelp to identify an error in buggy code. Examples of queries for Tasks 2 and 3 included:

- *I want to use this code to determine if there is a repeated value in the input array, but there seems to be a bug*
- *it contains a bug*

An example response to the query “it contains a bug”, when the code for Task 2 (see Figure 3) was included verbatim, is the output shown in Figure 5.

Evaluations of correctness and helpfulness: After each of these tasks, students were asked to evaluate the quality of the response that was generated by CodeHelp, with respect to *correctness* and *helpfulness*. Specifically, students responded to the following two Likert-scale items for each of the three programming tasks:

For this task, you will explore the feedback generated by LLM-TA if it is asked to *debug code*. Below is a function definition that should test whether or not an array contains any repeated values (i.e. values that appear more than once). However, the definition contains a bug. Even if you can see the bug, you should still submit the code below to LLM-TA and critique the output.

```
int IsRepeated(int values[], int numValues)
{
    for (int i = 0; i < numValues; i++) {
        for (int j = i; j < numValues; j++) {
            if (values[i] == values[j]) {
                return 1;
            }
        }
    }
    return 0;
}
```

Copy the code above (i.e. the function definition) and provide it as input to LLM-TA. You should construct a short natural language description asking it to *debug the code for this function*.

Figure 3: Task 2 – debugging the IsRepeated() function

The code for the function is below, however it contains a bug. Even if you can see the bug, you should still submit the code below to LLM-TA and critique the output.

```
int SurroundingSum(int values[10][10], int row, int col)
{
    int i, j;
    int sum = 0;
    for (i = row-2; i <= row+2; i++) {
        for (j = col-2; j <= col+2; j++) {
            if (i != row || j != col) {
                sum += values[i][j];
            }
        }
    }
    return sum;
}
```

int sum = SurroundingSum(values, 4, 4);

0	1	2	3	4	5	6	7	8	9
0	3	2	1	4	6	5	0	2	4
1	7	3	2	1	4	6	5	0	2
2	1	7	3	2	1	4	6	5	0
3	0	1	7	3	2	1	4	6	5
4	4	0	1	7	3	2	1	4	6
5	8	4	0	1	7	3	2	1	4
6	1	8	4	0	1	7	3	2	1
7	2	1	8	4	0	1	7	3	2
8	6	2	1	8	4	0	1	7	3
9	6	2	1	8	4	0	1	7	3

Sum = 72

Figure 4: Task 3 – debugging the SurroundingSum() function

The issue with your function lies in the nested for loop. You're comparing each element in the array with itself and all the subsequent elements. This means that each element will always find itself as a duplicate because you're starting the inner loop from 'i' instead of 'i + 1'.

In your current implementation, when 'i' and 'j' are the same, 'values[i]' is equal to 'values[j]', and your function will return '1' indicating that a duplicate is found, even if there are no duplicates in the array.

To fix this, you should start the inner loop from 'i + 1' instead of 'i'. This way, you're comparing each element with all the elements that come after it, not including itself. Remember, when dealing with nested loops, especially in cases where you're comparing elements in an array, it's important to carefully consider where each loop starts and ends to avoid comparing an element with itself.

Keep going, you're doing great!

Figure 5: Example response from CodeHelp to Task 2.

- (1) The response from CodeHelp is **correct** (i.e. it does not include any false or inaccurate content)
- (2) The response from CodeHelp is **helpful** (i.e. is a useful response for a student asking this question)

3.3 Reflections on desirable characteristics

After evaluating the feedback generated by CodeHelp to the three programming tasks, students were asked to think about and describe the characteristics they felt were important for a digital TA used in programming courses. Specifically, they were asked to answer the following: *Now that you have observed feedback from CodeHelp in response to requests for writing and debugging code, answer the following reflection question: What do you think are the most important characteristics for an automated programming assistant (like CodeHelp) if it is to be suitable for use in a programming course?*

The deadline for completing the lab tasks and for submitting the reflection was midnight on the evening of October 16th. After this date, students were invited to use CodeHelp optionally during the week following the lab when they were working on their end-of-course projects (worth 12% of their final grade). Students were permitted to use CodeHelp to ask any questions related to the final project, which was due at midnight on the evening of October 21st.

3.4 Data analysis

We report student interactions with CodeHelp in two ways. First, we perform a quantitative analysis of the frequency with which students interact with the tool over the duration of the study (9–21 October) and we summarize student ratings of the correctness and accuracy of the responses generated by CodeHelp.

Second, our qualitative analysis of the open-response reflection data follows the guidelines for reflexive thematic analysis outlined by Braun and Clarke [3]. In this study, three of the authors independently coded a sample of responses with succinct labels, and subsequently met to discuss the codes and to develop higher-level themes². When presenting the results, we report the most common themes and illustrate these with examples of student responses.

4 RESULTS

4.1 Interactions with the tool

Figure 6 illustrates when, over the two week period of the study (9–21 October), students submitted queries to CodeHelp. A total of 6,051 queries were made by 813 students. The intense period of activity, leading up to midnight on October 16th, corresponds to the deadline for the lab activity. Notably, there was considerable usage of CodeHelp after October 16th, when student interactions with the tool were ungraded.

Figure 7 shows the percentage of all active students using CodeHelp each day (left-hand axis), along with the total number of queries to CodeHelp (right-hand axis). After the lab deadline (16 October), the percentage of students using CodeHelp remained fairly stable (although was trending downwards), but the number of daily queries increased as the project deadline drew nearer. Given that this corresponds to voluntary use of CodeHelp, it suggests students saw value in the tool for supporting their coursework.

Figure 8 summarizes student responses to the Likert-scale items regarding the correctness and helpfulness of CodeHelp’s responses to the three questions on the lab. A large majority of the students either “Strongly agree” or “Agree” that the responses generated by CodeHelp were both correct and helpful.

²<https://www.thematicanalysis.net/doing-reflexive-ta/>

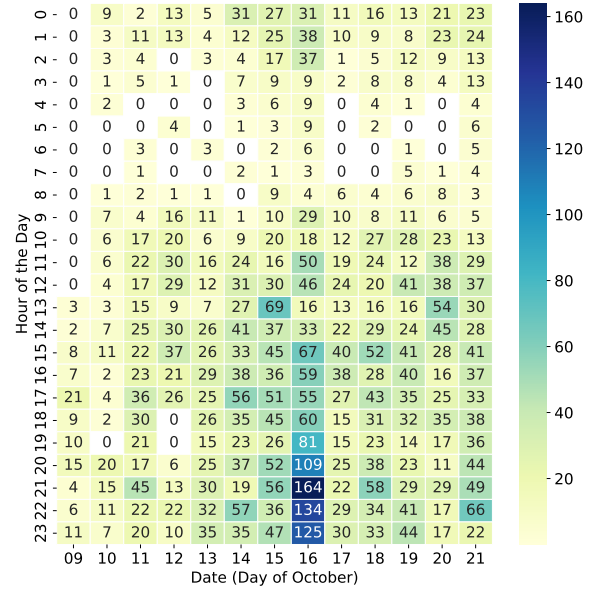


Figure 6: Queries submitted per hour (9-21 October). Midnight deadlines 16 October (Lab) and 21 October (Project).

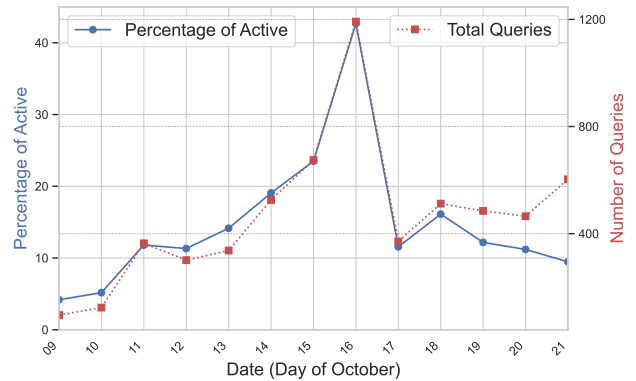


Figure 7: Percentage of students (out of all active) submitting queries per day and total number of queries.

4.2 Desired characteristics

Our qualitative analysis of student responses on the important characteristics for a digital TA revealed three key themes: **Scaffolding** (i.e., *how* student learning should be supported); **Appropriateness** (i.e., the *nature and content* of explanations and feedback); and **User Experience** (i.e., the *values* that students express when considering their learning experience). Each theme highlights a crucial aspect of the educational support system students would expect from an AI Teaching Assistant.

4.2.1 Scaffolding. Students reported wanting to understand the content and felt that explanations that focused on process (*how*) and foundational knowledge (*why*) were key features. They did not want to be directly provided with the solutions to the problems,

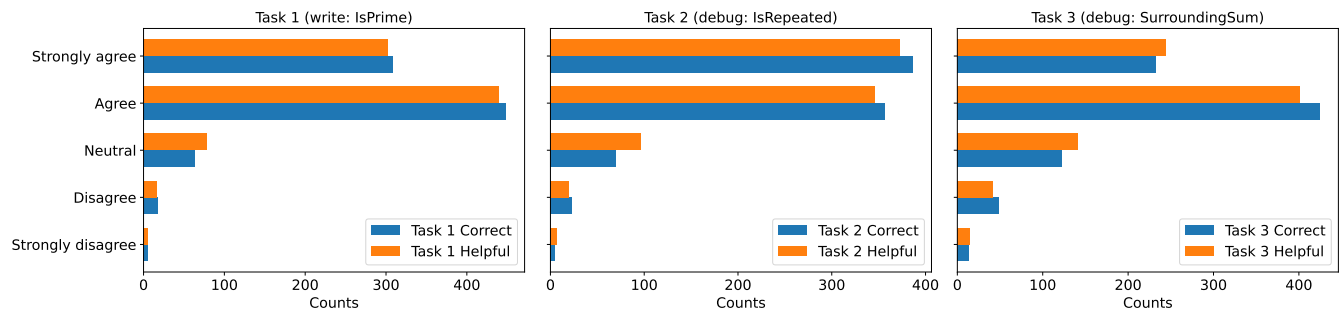


Figure 8: Student feedback on the correctness and helpfulness of responses from CodeHelp to the three lab questions.

but rather, they wanted to be taught how to obtain the solutions themselves (agency) to develop deep understanding. The student reflections included, among others, the following:

“One key idea is that it shouldn’t supply code.”

“Therefore the assistant must make sure to be helpful without directly writing code or debugging code for the user by giving ideas and explaining logic and asking relevant questions to help the user understand.”

Students also described the importance of support for focusing on relevant code sections:

“Like suggesting changing a small portion ... as people can focus on one section of the code.”

Some students emphasized the benefits of having support in problem-solving guidance:

“... useful for understanding code or a complex algorithm as they break the problem down into simpler steps to understand.”

“This may be done by asking the student questions instead of giving answers, such that the questions provided give the student a clue as to what to do.”

Several students talked about the value of multiple exemplars which is a good pedagogical practice:

“... beneficial to have an assistant that presents a few different ways of solving an issue, to make students reflect more, but also to make sure there is a solution that could be implemented with varying levels of coding experience.”

“The ability to provide explanations on how to write code that fulfills the specifications in various forms so that the user can expand their coding repertoire.”

Finally, they also talked about breaking up explanations in ways that may reduce cognitive effort:

“For writing of code I think the AI should break it down into simple steps for the student to focus on and write rather than writing the code for the student, as this will help the student learn.”

“The tool must also provide good explanations instead of just solutions and should be able to break down unfamiliar concepts in simple terms.”

4.2.2 Appropriateness. The second theme that emerged is the need for the presented concepts and explanations as well as language expressions to be appropriate for the level of the audience.

“... [an] important characteristic would be understandability. If the feedback provided was not written in a well-structured manner, this would require a lot of effort for us to slowly understand and read ...”

“... It should be detailed enough that the person asking will be able to understand the information provided and how to fix a certain issue ...”

“Not everyone will be at the same level of programming as others. Hence, the explanations should be customised for everyone so that users of different levels of coding experience can find this assistant useful.”

Many students clearly preferred explanations in plain language, avoiding complex jargon:

“conciseness is important... The information provided [by the model] should be specific to the problem provided and not information based on general topics.”

“I think the most important characteristic is providing help in a way that is easier to understand. This means organising hints in a bulleted list, using simple language...”

Further, the content should be constrained to the topics that have been covered in the course, and which are appropriate for beginners.

“and for more complex problems, providing snippets of code that beginners may not be familiar with yet.”

“... showing solutions using only methods taught in class... Chat-GPT for example fails at this often... [it] may suggest far more complex solutions that a beginner cannot yet understand.”

4.2.3 User Experience. The final theme encompasses the values that students describe as important for their experience when using the tool for learning. They emphasise the importance of agency:

“broad cues as help could be more effective in letting students ‘find’ their own solutions from these cues.”

“help the student figure out their own way of solving the problem.”

“It should also be interactive, allowing students to try things for themselves whilst providing feedback and error analysis.”

Students wanted rapid responses with support always available:

“Sometimes you can’t always get help or assistance from someone who is experienced in coding and can help you, so to have a 24/7 assistance that can achieve the same level as a tutor/teacher is really helpful.”

“helps explain codes without the help of a tutor, helps the student to get help all the time everywhere.”

Most importantly, the responses from the tool must be correct. Trustworthiness and precision in the feedback are crucial for students to be confident in using such tools for educational purposes:

“I think the accuracy is very important. Having an automated programming assistant that is very accurate is essential.”

In summary, the thematic analysis indicates that students seek to enhance their learning by ensuring they understand the content and develop their critical thinking and problem-solving skills. They want an educational aid rather than a solution provider. They desire a tool that helps them understand their coding errors and guides them to correct those mistakes, all communicated in clear and concise language and with accurate feedback that they can trust.

5 DISCUSSION

The use of AI Teaching Assistants (TAs) is likely to increase, especially given their cost effectiveness relative to human TAs. In this study, we used the most expensive model from OpenAI at the time (GPT-4). The total cost for handling 6,051 queries spread across all hours of the day over 2 weeks was US\$510, roughly equivalent to a little over 30 hours of human TA time (assuming a typical rate of US\$15 per hour). Such tools may not entirely replace human TAs, and we do not advocate for that, but they may be a useful way to augment human TA support, especially by answering common queries outside of regular working hours—a time when our prototype experienced substantial student usage. Usage spiked dramatically leading up to the deadline, with 20% of all queries being submitted on the 16th October, and many of those in the hours immediately before the lab deadline. Future research could investigate the impact of always available support on student procrastination, given previous findings linking automatic grading to increased procrastination [1].

Our qualitative results align with findings from previous research, highlighting a trend where students weigh the convenience of help resources against perceived learning benefits [10]. Notably, students preferred shorter responses that facilitated their problem-solving process in a structured manner rather than receiving solutions directly. This also aligns with students’ responses about in-person TAs [18], however this same prior work found that students sometimes want direct answers when seeking help close to a deadline. Future work could examine if student opinions change (or if they would opt for just a solution) when near a deadline. It’s also possible that students felt more comfortable expressing maladaptive learning behaviors (just wanting solutions) in the 1-1 interviews conducted by Lim et al. than in our questionnaire.

Encouragingly, this explicit desire for solutions to remain hidden suggests that students want to use AI TAs for learning, not to obtain solutions. This contrasts with the concerns expressed by academics about use of AI by students to cheat [25, 32, 37]. Prior work provides evidence that students are most likely to cheat when they run out of time, when the workload is too high, and when assignments are too hard [31]. AI TAs may provide a higher level of individual support to students that may reduce time spent waiting for feedback, hints, and other teaching support humans provide. A recent study also highlighted the on-demand nature of help from

AI tools as a primary motivator for student use [10]. AI TAs that provide immediate and personalized feedback may therefore *reduce* academic misconduct as students can get assistance promptly and overcome some of the pressures that lead to cheating behaviours.

However, it is crucial for instructors to continue to emphasize the purpose and value of learning activities and assignments. In a recent interview study, some students claimed they would resort to using AI tools to complete ‘busy work’, referring to activities perceived as lacking educational value [37]. So, while clearly the design of AI TAs is critical, instructors also play a pivotal role in ensuring ethical use of AI TAs in the learning process.

5.1 Design Recommendations

5.1.1 Scaffold a Learning Process that Preserves Students’ Agency. Students expressed the desire to independently arrive at a solution rather than being provided with solutions directly. This preference aligns with recent research outside of computing education which demonstrated that users are more likely to engage with AI tools when they feel a sense of ownership over the process and outcomes [11, 23, 36]. Exploring the impact of AI Teaching Assistants on students’ agency in their learning process would be a promising area for future research. While students want their agency to be preserved, they still appreciated hints and guidance in the form of procedural scaffolding. Rather than receiving direct instructions about how to fix their code, students wanted to have their attention directed to the area within the code that needed attention or help with understanding the problem they are trying to solve.

5.1.2 Tailor Scaffolding and Responses to Students’ Expertise. Students wanted responses to be aligned with their knowledge and abilities. Students highlighted issues with current tools like ChatGPT, which fail to account for their expertise level, resulting in confusing and unhelpful programming patterns. However, they also expressed interest in being exposed to new patterns and examples that could challenge and expand their abilities. AI Teaching Assistants should therefore maintain user models that can be used to guide process and personalize responses. In addition, students advocated for responses to be clearly and concisely presented, such as by using bullet points. This aligns with best practice in feedback design [14]. These findings underscore the importance of providing well-organized responses that are easily comprehensible.

6 CONCLUSION

In this study, we explored key implications arising from students’ reflections about using AI-assisted learning tools in the domain of computing education. Our findings underscore two key points. First, students not only enjoyed using the AI TA but also engaged with it extensively, especially late at night leading up to the lab deadline. Second, students desired more than mere answers; seeking agency in their learning process and emphasizing the need for scaffolding to guide them through the problem-solving process, and to help them generate their own solutions.

ACKNOWLEDGMENTS

The authors would like to thank Mark Liffiton for developing the Gen-Ed framework and CodeHelp, and to both Mark Liffiton and Brad Sheese for their contributions to prior research on CodeHelp.

REFERENCES

- [1] Tapio Auvinen. 2015. Harmful Study Habits in Online Learning Environments with Automatic Assessment. In *2015 International Conference on Learning and Teaching in Computing and Engineering*. IEEE, 50–57.
- [2] Rishabh Balse, Bharath Valaboju, Shreya Singhal, Jayakrishnan Madathil Warriem, and Prajish Prasad. 2023. Investigating the Potential of GPT-3 in Providing Feedback for Programming Assessments. In *Proc of the 2023 Conference on Innovation and Technology in CS Education V. 1* (Turku, Finland) (ITiCSE 2023). ACM, NY, USA, 292–298. <https://doi.org/10.1145/3587102.3588852>
- [3] Virginia Braun and Victoria Clarke. 2006. Using Thematic Analysis in Psychology. *Qualitative Research in Psychology* 3, 2 (2006), 77–101.
- [4] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2022. CodeT5: Code Generation with Generated Tests. <http://arxiv.org/abs/2207.10397> arXiv:2207.10397 [cs].
- [5] Paul Denny, Viraj Kumar, and Nasser Giacaman. 2023. Conversing with Copilot: Exploring Prompt Engineering for Solving CS1 Problems Using Natural Language. In *Proc of the 54th ACM Technical Symposium on CS Education V. 1*. ACM, Toronto ON Canada, 1136–1142. <https://doi.org/10.1145/3545945.3569823>
- [6] Paul Denny, James Prather, Brett A. Becker, James Finnie-Ansley, Arto Hellas, Juho Leinonen, Andrew Luxton-Reilly, Brent N. Reeves, Eddie Antonio Santos, and Sami Sarsa. 2024. Computing Education in the Era of Generative AI. *Commun. ACM* 67, 2 (Jan 2024), 56–67. <https://doi.org/10.1145/3624720>
- [7] James Finnie-Ansley, Paul Denny, Brett A. Becker, Andrew Luxton-Reilly, and James Prather. 2022. The Robots are Coming: Exploring the Implications of OpenAI Codex on Introductory Programming. In *Proceedings of the 24th Australasian Computing Education Conference*. 10–19. <https://doi.org/10.1145/3511861.3511863>
- [8] Arto Hellas, Juho Leinonen, Sami Sarsa, Charles Koutchene, Lilja Kujanpää, and Juha Sorva. 2023. Exploring the Responses of Large Language Models to Beginner Programmers' Help Requests. In *Proceedings of the 2023 ACM Conference on International Computing Education Research - Volume 1* (Chicago, IL, USA) (ICER '23). ACM, New York, NY, USA, 93–105. <https://doi.org/10.1145/3568813.3600139>
- [9] Irene Hou, Owen Man, Sophia Mettillie, Sebastian Gutierrez, Kenneth Angelikas, and Stephen MacNeil. 2024. More Robots are Coming: Large Multimodal Models (ChatGPT) can Solve Visually Diverse Images of Parsons Problems. In *Proc 26th Australasian Comp Ed Conf* (Sydney, Australia) (ACE '24). ACM, New York, NY, USA, 29–38. <https://doi.org/10.1145/3636243.3636247>
- [10] Irene Hou, Sophia Mettillie, Owen Man, Zhuo Li, Cynthia Zastudil, and Stephen MacNeil. 2024. The Effects of Generative AI on Computing Students' Help-Seeking Preferences. In *Proc 26th Australasian Comp Ed Conf* (Sydney, Australia) (ACE '24). ACM, NY, USA, 39–48. <https://doi.org/10.1145/3636243.3636248>
- [11] Ziheng Huang, Kexin Quan, Joel Chan, and Stephen MacNeil. 2023. CausalMapper: Challenging designers to think in systems with Causal Maps and Large Language Model. In *Proc of the 15th Conf on Creativity and Cog. ACM*, 325–329.
- [12] Breanna Jury, Angela Lorusso, Juho Leinonen, Paul Denny, and Andrew Luxton-Reilly. 2024. Evaluating LLM-generated Worked Examples in an Introductory Programming Course. In *Proc 26th Australasian Comp Ed Conf* (Sydney, Australia) (ACE '24). ACM, NY, USA, 77–86. <https://doi.org/10.1145/3636243.3636252>
- [13] Majeed Kazemitabaar, Justin Chow, Carl Ka To Ma, Barbara J. Ericson, David Weintrop, and Tovi Grossman. 2023. Studying the Effect of AI Code Generators on Supporting Novice Learners in Introductory Programming. In *Proc of the 2023 CHI Conf on Human Factors in Computing Systems* (Hamburg, Germany) (CHI '23). ACM, NY, USA, Article 455, 23 pages. <https://doi.org/10.1145/3544548.3580919>
- [14] Markus Krause, Tom Garnicarz, Jiaojiao Song, Elizabeth M Gerber, Brian P Bailey, and Steven P Dow. 2017. Critique style guide: Improving crowdsourced design feedback with a natural language model. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. 4627–4639.
- [15] Sophia Krause-Levy, Rachel S. Lim, Ismael Villegas Molina, Yingjun Cao, and Leo Porter. 2022. An Exploration of Student-Tutor Interactions in Computing. In *Proc of the 27th ACM Conf on Innovation and Tech in CS Education Vol. 1* (Dublin, Ireland) (ITiCSE '22). ACM, NY, USA, 435–441. <https://doi.org/10.1145/3502718.3524786>
- [16] Juho Leinonen, Paul Denny, Stephen MacNeil, Sami Sarsa, Seth Bernstein, Joanne Kim, Andrew Tran, and Arto Hellas. 2023. Comparing Code Explanations Created by Students and Large Language Models. In *Proc of the 2023 Conference on Innovation and Technology in CS Education V. 1* (Turku, Finland) (ITiCSE 2023). ACM, New York, NY, USA, 124–130. <https://doi.org/10.1145/3587102.3588785>
- [17] Mark Liffiton, Brad E Sheese, Jaromir Savelka, and Paul Denny. 2024. CodeHelp: Using Large Language Models with Guardrails for Scalable Support in Programming Classes. In *Proc of the 23rd Koli Calling International Conference on Comp Ed Research* (Koli, Finland) (Koli Calling '23). ACM, NY, USA, Article 8, 11 pages. <https://doi.org/10.1145/3631802.3631830>
- [18] Rachel S. Lim, Sophia Krause-Levy, Ismael Villegas Molina, and Leo Porter. 2023. Student Expectations of Tutors in Computing Courses. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1* (Toronto, Canada) (SIGCSE 2023). ACM, NY, USA, 437–443. <https://doi.org/10.1145/3545945.3569766>
- [19] Stephen Macneil, Paul Denny, Andrew Tran, Juho Leinonen, Seth Bernstein, Arto Hellas, Sami Sarsa, and Joanne Kim. 2024. Decoding Logic Errors: A Comparative Study on Bug Detection by Students and Large Language Models. In *Proc 26th Australasian Comp Ed Conf* (Sydney, Australia) (ACE '24). ACM, NY, USA, 11–18. <https://doi.org/10.1145/3636243.3636245>
- [20] Stephen MacNeil, Andrew Tran, Arto Hellas, Joanne Kim, Sami Sarsa, Paul Denny, Seth Bernstein, and Juho Leinonen. 2023. Experiences from Using Code Explanations Generated by Large Language Models in a Web Software Development E-Book. In *Proc of the 54th ACM Tech Sym on CS Education V. 1* (SIGCSE 2023). ACM, NY, USA, 931–937. <https://doi.org/10.1145/3545945.3569785>
- [21] Julia M Markel and Philip J Guo. 2021. Inside the Mind of a CS Undergraduate TA: A firsthand account of undergraduate peer tutoring in computer labs. In *Proc of the 52nd ACM Tech Symposium on CS Education*. 502–508.
- [22] Nhan Nguyen and Sarah Nadi. 2022. An empirical evaluation of GitHub copilot's code suggestions. In *Proc of the 19th Int. Conf. on Mining Software Repositories*. ACM, Pittsburgh Pennsylvania, 1–5. <https://doi.org/10.1145/3524842.3528470>
- [23] Changhoon Oh, Jungwoo Song, Jinhan Choi, Seonghyeon Kim, Sungwoo Lee, and Bongwon Suh. 2018. I Lead, You Help but Only with Enough Details: Understanding User Experience of Co-Creation with Artificial Intelligence. In *Proc of the 2018 CHI Conf on Human Factors in Computing Systems* (Montreal, Canada) (CHI '18). ACM, NY, USA, 1–13. <https://doi.org/10.1145/3173574.3174223>
- [24] Zachary A. Pardos and Shreya Bhandari. 2023. Learning gain differences between ChatGPT and human tutor generated algebra hints. arXiv:2302.06871 [cs.CY]
- [25] James Prather, Paul Denny, Juho Leinonen, Brett A. Becker, Ibrahim Albluwi, Michelle Craig, et al. 2023. The Robots Are Here: Navigating the Generative AI Revolution in Computing Education. In *Proc of the 2023 Working Group Reports on Innovation and Technology in CS Education* (Turku, Finland) (ITiCSE-WGR '23). ACM, New York, NY, USA, 108–159. <https://doi.org/10.1145/3623762.3633499>
- [26] James Prather, Brent N. Reeves, Paul Denny, Brett A. Becker, Juho Leinonen, Andrew Luxton-Reilly, Garrett Powell, James Finnie-Ansley, and Eddie Antonio Santos. 2023. "It's Weird That it Knows What I Want": Usability and Interactions with Copilot for Novice Programmers. *ACM Trans. Comput.-Hum. Interact.* 31, 1, Article 4 (Nov 2023), 31 pages. <https://doi.org/10.1145/3617367>
- [27] Emma Riese, Madeleine Lorás, Martin Ukrop, and Tomáš Effenberger. 2021. Challenges Faced by Teaching Assistants in Computer Science Education Across Europe. In *Proc 26th ACM Conf on Inn and Tech in CS Ed* (Virtual Event, Germany) (ITiCSE '21). ACM, NY, USA, 547–553. <https://doi.org/10.1145/3430665.3456304>
- [28] Sami Sarsa, Paul Denny, Arto Hellas, and Juho Leinonen. 2022. Automatic Generation of Programming Exercises and Code Explanations Using Large Language Models. In *Proceedings of the 2022 ACM Conference on International Computing Education Research V.1*. ACM, Lugano and Virtual Event Switzerland, 27–43. <https://doi.org/10.1145/3501385.3543957>
- [29] Jaromir Savelka, Arav Agarwal, Marshall An, Chris Bogart, and Majd Sakr. 2023. Thrilled by Your Progress! Large Language Models (GPT-4) No Longer Struggle to Pass Assessments in Higher Education Programming Courses. In *Proceedings of the 2023 ACM Conference on International Computing Education Research V.1* (ICER 2023). ACM. <https://doi.org/10.1145/3568813.3600142>
- [30] Jaromir Savelka, Arav Agarwal, Christopher Bogart, Yifan Song, and Majd Sakr. 2023. Can Generative Pre-trained Transformers (GPT) Pass Assessments in Higher Education Programming Courses?. In *Proc of the 2023 Conference on Innovation and Technology in CS Education V. 1* (Turku, Finland) (ITiCSE 2023). ACM, New York, NY, USA, 117–123. <https://doi.org/10.1145/3587102.3588792>
- [31] Judy Sheard, Angela Carbone, and Martin Dick. 2003. Determination of factors which impact on IT students' propensity to cheat. In *Proceedings of the Fifth Australasian Conference on Computing Education - Volume 20* (Adelaide, Australia) (ACE '03). Australian Computer Society, Inc., AUS, 119–126.
- [32] Judy Sheard, Paul Denny, Arto Hellas, Juho Leinonen, Lauri Malmi, and Simon. 2024. Instructor Perceptions of AI Code Generation Tools - A Multi-Institutional Interview Study. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1* (Portland, OR, USA) (SIGCSE 2024). ACM, New York, NY, USA, 1223–1229. <https://doi.org/10.1145/3626252.3630880>
- [33] Aaron J. Smith, Kristy Elizabeth Boyer, Jeffrey Forbes, Sarah Heckman, and Ketan Mayer-Patel. 2017. My Digital Hand: A Tool for Scaling Up One-to-One Peer Teaching in Support of Computer Science Learning. In *Proc of the 2017 ACM SIGCSE Technical Symposium on CS Education* (Seattle, Washington, USA) (SIGCSE '17). ACM, NY, USA, 549–554. <https://doi.org/10.1145/3017680.3017800>
- [34] Margaret Smith, Yujie Chen, Rachel Berndtson, Kristen M Burson, and Whitney Griffin. 2017. "Office Hours Are Kind of Weird": Reclaiming a Resource to Foster Student-Faculty Interaction. *InSight: A Journal of Scholarly Teaching* 12 (2017), 14–29.
- [35] Andrew Tran, Kenneth Angelikas, Egi Rama, Chiku Okechukwu, David H Smith, and Stephen MacNeil. 2023. Generating Multiple Choice Questions for Computing Courses Using Large Language Models. In *2023 IEEE Frontiers in Education Conference (FIE)*. IEEE Computer Society, 1–8.
- [36] Ann Yuan, Andy Coenen, Emily Reif, and Daphne Ippolito. 2022. Wordcraft: story writing with large language models. In *27th International Conference on Intelligent User Interfaces*. 841–852.
- [37] Cynthia Zastudil, Magdalena Rogalska, Christine Kapp, Jennifer Vaughn, and Stephen MacNeil. 2023. Generative AI in Computing Education: Perspectives of Students and Instructors. In *2023 IEEE Frontiers in Education Conference (FIE)*. IEEE Computer Society, 1–9.